CS3600 Spring 2026 Tournament Rules

1 Basics

Each agent controls a worker on an 8x8 chessboard. The agents take turns making moves. After both have had 40 turns, the game is terminated.

Each agent gets a total of 4 minutes to come up with its moves. (Not 4 minutes for each move. A total of 4 minutes for all your moves. The game will not last longer than 8 minutes.) If a player runs out of time, they lose the game.

The agent with the most points when the game terminates is the winner. You win points through these activities:

- **Priming**: When you paint a square with glue, you gain 1 point.
- **Carpeting**: When you carpet a line of n primed squares, you gain points based on the following table. Either player can step into a carpeted square.

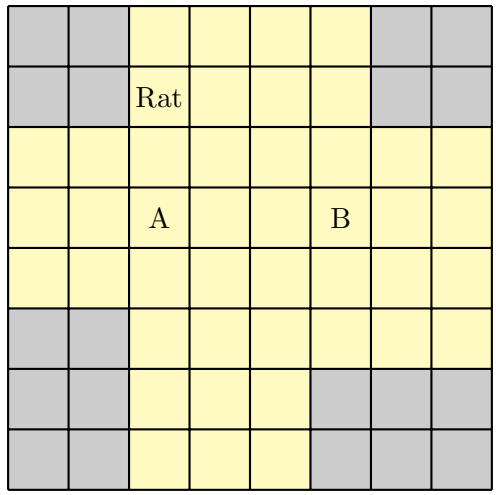
n	1	2	3	4	5	6	7
Points	-1	2	4	6	10	15	21

- **Catching the Rat**: If you guess the square where the rat is hidden, you gain 4 points. If your guess the rat location incorrectly, you lose 2 points.

2 Start

Each corner of the map will have blocked squares randomly arranged in either a 3x2, 2x3, or 2x2 rectangle.

Player A moves first. Player B moves second. Both players spawn within the center 4x4 squares in horizontally mirrored positions.



There is a rat hidden in one of the squares.

You are given a 64x64 matrix where each cell (x, y) is mapped to a flat index $i = y \cdot 8 + x$. The matrix defines the rat's movement probabilities:

$$T(i, j) = P(\text{moving from cell i to cell j})$$

Naturally, for any given i, only five entries can be non-zero (representing the rat staying in place or moving to one of the four adjacent cardinal neighbors) and the probabilities always sum to 1.0. At the edge, the probability of going off the edge will always be zero. The rat can go under blocked squares.

The T table is different for every game, but it stays constant for the whole game.

Where does the rat start? The rat is given a headstart: It is placed in square $(0, 0)$ and moves freely (but silently) for 1000 moves (following the T table mentioned above). Each new rat is given the same 1000 moves before the game resumes.

3 Play

moves

Each turn, the agent either:

- picks a move to perform
- guesses the location of the rat.

Two workers cannot occupy the same square. 4 moves

The worker can take any of the following steps in any of the four cardinal directions:

1. A **Plain Step** simply changes its position. You cannot plain step onto a primed square.

2. A **Prime Step** can only happen on a square that is not primed or carpeted. When the worker departs the square, it becomes primed. You cannot prime step onto a primed square.
3. A **Carpet Roll** of length k moves the worker in a straight line over k contiguous primed squares, starting from a square adjacent to the worker's current position, carpeting each square. The worker ends on the k-th (final) square.

Alternatively, the worker may take a **Search Move**, where it searches any cell on the map for the rat (but the worker itself does not move). If the worker finds the rat, it gains 4 points. If the worker does not find the rat in the searched square, you lose 2 points.

4 The Rat

There is always a rat under the floor of one of the squares. If you guess where the rat is, you win 4 points. If you guess wrong, you lose 2 points.

Before each turn, the rat moves one square in one of the cardinal directions (or stays) according to the T table and makes a noise.

The rat can make the following noises: `squeak`, `scratch`. The probability of the noise depends on the type of the floor the rat is under.

	Squeak	Scratch	Squeal
Blocked	0.5	0.3	0.2
Space	0.7	0.15	0.15
Primed	0.1	0.8	0.1
Carpet	0.1	0.1	0.8

Reported measure	probability
One less than actual	0.12
Correct	0.7
One more than actual	0.12
Two more than actual	0.06

On your worker's turn, it will hear the rat. Besides the nature of the sound, it will also estimate the manhattan distance to the rat's square. However, the distance measure is not exact (see table above). each turn: given rat position estimate

Your worker will never estimate less than zero. (If we roll the dice, and the estimate would be less than zero, you will receive an estimate of zero.)

As soon as the rat is captured, a new rat is spawned. It is allowed to run for 1000 steps before the next player is given a turn.

If your opponent guesses on their turn, you will be told what they guessed and whether they were right or not. important

5 Termination conditions

Here is the definitive list of all the ways that the game can end and who wins:

- A worker makes an invalid move. The game is terminated and the responsible agent loses.
- An agent runs out of time. The game is terminated and that agent loses.
- Both agents have made all their moves. The game is terminated and the worker with the most points wins.

6 Environment

Each agent will be submitted as a python script called `agent.py` in a zipfile. That zipfile can contain code or data that you created, but the total size of the zip file may not exceed 200 MB. (Code or data created by anyone outside of your team is forbidden, but you are allowed to use code that was created by an LLM prompted by someone on your team.) The agent may not make any requests via the network. It may not read or write outside of its working directory. Python 3.12 and the following libraries will be installed on the machine: numpy, PyTorch, JAX, FLAX, Plyvel, and Scikit-learn. The tournament machine will be a x86_64 linux machine.

The state of the game arrives in an instance of a class called `Board` and a tuple containing (noise, estimated distance). The positions on the board are (x, y) tuples. $(0, 0)$ is considered the upper left corner. Together, they contain the following information:

- Your Worker: Worker Object
  - Your score: int
  - Your position: tuple (int,int)
  - Your turns (including the one you are about to take) remaining : int
  - Your total time remaining: float
- Opponent's Worker: Worker Object
  - Same as above
- Board masks (64 bits each):
  - Blocked squares
  - Space squares
  - Primed squares
  - Carpeted squares
- Opponent's search location and result: ((int, int), bool)
  - (None, False) if your opponent did not guess
- Your search location and result: ((int, int), bool)
  - (None, False) if your opponent did not guess

You will return:

- A move: Move object

Documentation on how moves are represented are located in `move.py`.

You want to make a state/action game tree? Look in `board.py` at `forecast_move()` `and reverse_perspective()`.

Finally, you will receive a function that you can call at anytime to get your remaining time.

7 Development

Unzip the provided zipfile anywhere on your hard drive. The source for the game system is on there, so you will be able to test your agent locally.

You will be working with a partner. It is strongly suggested that you keep your project in a git repository that you both have access to. This will make collaboration much easier. Your bots should be placed in a folder under "3600-agents". An example bot, Yolanda, is provided for reference.

To run the game with Yolanda as both player A and player B, you can run the following:

```
python3 engine/run_local_agents.py Yolanda Yolanda
```

You should see a log of the two Yolandas playing the game. The history of the game will be in `3600-agents/matches/Yolanda_Yolanda_0.json`. (You probably don't want to add that to your repository. You might want to add matches to your `.gitignore` file.)

Please report any bugs you find to the ed discussion.

7.1 Multiple Files

Python has rules about what directory you can import files from. If you partition your agent into several `.py` files, you will probably bump into these rules.

The fix is to create an `__init__.py` file in the same directory as your `agent.py` file. For example, if my `agent.py` file wanted to import my `rat_belief.py` file, I would put this in an `__init__.py` file:

```
from .agent import PlayerAgent
from . import rat_belief
```

Then, in `agent.py`, I could import the `RatBelief` class in `rat_belief.py` like this:

```
from .rat_belief import RatBelief
```

7.2 LLM Usage and Plagiarism

AI usage along the likes of ChatGPT, Gemini, Claude, or Deepseek is allowed. Coding agents are permitted. However, past students have noted hallucinations in implementations of game playing and AI algorithms. It is your responsibility to understand and validate any code that you prompt an LLM to write.

Rather than simply asking LLMs to write code for you, a strategy that will help deepen your understanding of your own program is to attempt your first pass at an algorithm without AI, then ask LLMs to validate/criticize your implementation.

Collaborating with human collaborators outside your team is not permitted.

7.3 Infrastructure and Recommendations

Developing AI algorithms is also about designing good infrastructure for developing and testing your algorithms. The provided running scripts runs one match between two bots. However, it may interest you to understand performance across multiple games, since the board is stochastic. Since this can take a while, you could also write parallelization, batch scripts, and any other strategy you find appropriate to accelerate how quickly you can iterate. For example, if you develop a program that requires a heuristic with several components, and are tunable coefficients determining their balance, you might want to run 50 matches at 10 different settings of hyperparameters.

8 Submitting

You feel ready to see how your agent will do in competition? Zip up the agent directory. For example, you might zip up the `Yolanda` directory into `Yolanda.zip`. It should contain a directory with an `agent.py` file inside.

Go to `https://bytefight.org/` Register. (There is a decent chance the "Verify Code" email will be sent to your Spam folder.)

Create a team and add both members of the team to it. (Please use the same email address that we have in Banner/Canvas for you. This will increase the probability that you will get credit for your work.)

Upload the zipfile. Pick an agent to scrimmage against. Good luck!

9 Grades and Awards

Whatever you have uploaded/activated at 11:59pm on April 19th 2026 will be used in the competition and determine your ranking and your grade.

ELO scores are used to determine the ranking. Everyone's ELO will be reset to 1500 when the final submissions are locked in and the final tournament begins.

For grading, we have reference agents:

- If you are above George in the final rankings you will get at least a 70%.
  - George is a no-lookahead bot that prefers to extend primes and roll carpet. It opportunistically searching for the rat when the expected value of a search is sufficiently high.
- If you are above Albert, you will get at least an 80%.
  - Albert is a expectiminimax-based bot with a *very* simple heuristic and a hidden markov model to track the rat.
- If you are above Carrie, you will get at least a 90%.

- Carrie uses the same expectiminimax and HMM structure as Albert, but uses a more advanced heuristic that takes into account an estimate of the potential of each cell and its distance from the bot.

Within each of these pools (70-80%, 80-90%, 90-100%), students will be graded on a scale based on their ELO versus the ELO of the reference bots. For example, if George has an ELO of 1700, Albert has an ELO of 1800, and your agent has an ELO of 1750, your grade will be 75

If you are the top team, you each get a $150 Amazon gift card. Also, you will be taken out for a nice lunch with the instructors.

Appendix A - Helpful Algorithms

1.1 Minimax

Discussed in *Adversarial Search and alpha-beta Pruning* lecture.

Minimax is a recursive decision-making algorithm for two-player, zero-sum games. It assumes that both players act optimally: one player tries to maximize the score, while the other tries to minimize it.

The game is represented as a tree of possible future states, where each node corresponds to a game position and each edge represents a legal move. Starting from the current state, Minimax explores all possible sequences of moves up to a fixed depth or until reaching a terminal game state. At a leaf node, a heuristic function is used to assign a numerical score to that position based on how favorable it is for the maximizing player.

These scores are then propagated back up the tree where max nodes choose the highest value and min nodes choose the lowest value. The move that leads to the best guaranteed outcome is chosen as the optimal move.

Naive minimax's search space explodes exponentially. However, there are several well known optimization techniques to reduce the search space by trimming branches of the tree:

- Alpha-beta Pruning
- Move Ordering
- Iterative Deepening
- Transposition Tables

There are countless other optimizations for minimax (especially from chess bot developers) which may or may not be applicable to our carpet game

Sebastian Lague has some interesting videos where he develops a minimax chess bot that you may find inspiration and optimization techniques from:
`https://www.youtube.com/playlist?list=PLFt_AvWsXl0cvHyu32ajwh2qU1i6hl77c`

1.2 Expectiminimax

Expectiminimax is a variation of minimax for games whose next state also contains an element of luck (i.e. rolling a dice). When evaluating a move that depends on random chance, expectiminimax takes the weighted average of the utility of all possible next state branches.

See: `https://en.wikipedia.org/wiki/Expectiminimax`

1.3 Monte Carlo Tree Search

Discussed in *Monte Carlo Tree Search and AlphaZero* lecture.

Monte Carlo Tree Search (MCTS) is a simulation-based search algorithm for decision-making in games. Instead of exhaustively exploring the full game tree like Minimax, it builds the tree gradually using random or semi-random simulations from the current position.

Each iteration of MCTS consists of four steps:

1. Selection: Traverse the tree using a strategy that balances exploring new moves and exploiting known good ones (ie UCB1)
2. Expansion: Add a new node when an unexplored state is reached
3. Simulation: Play out the game (often randomly) from that state to the end of the game
4. Backpropagation: Update statistics (win/loss/value) along the visited path

Over many simulations, MCTS builds a statistical estimate of which moves are strongest and focuses computation on the most promising parts of the search space.

1.4 AlphaZero

Discussed in *Monte Carlo Tree Search and AlphaZero* lecture.

In 2016, Google Deepmind developed an AI bot that famously beat world champion Lee Sedol at the game of Go, a game that AI researchers had long believed was too complex and had too large of a search space for computers to play effectively. There is a beautiful documentary on Alpha Go: `https://youtu.be/WXuK6gekU1Y?si=4mFrrReKj6PlKjux`

Alpha Go/Zero combines Monte Carlo Tree Search with deep learning to significantly improve search quality. Instead of relying on random rollouts, AlphaZero uses a neural network that evaluates game states and outputs:

- A policy: which moves are likely strong (often from historical data)
- A value: how good the position is for a player

These outputs guide MCTS by focusing search on promising moves and using the value estimate instead of long random simulations.. The system improves through self-play, where it repeatedly plays games against itself, trains on the results, and uses the updated network to guide future searches.

AlphaZero style bots are currently the strongest bots for many strategic games such as chess, go, and checkers but require significant amounts of data, training time, and compute. PACE will be a valuable resource for developing AlphaZero style bots. We may also release a dump of all game data later in the tournament (to be determined).

1.4.1 Hidden Markov Models

Discussed in *Markov Chains and Hidden Markov Models* lecture.

Hidden Markov Models (HMMs) are probabilistic models for tracking a hidden state that evolves over time based on noisy observations. They assume a system transitions between states according

to a known transition model and produces observations that depend on the current state.

An HMM maintains a belief distribution over all states and updates it by: Prediction: propagate belief through the transition model Update: reweight belief using the likelihood of the observation

This recursive process refines the estimate of the hidden state over time.